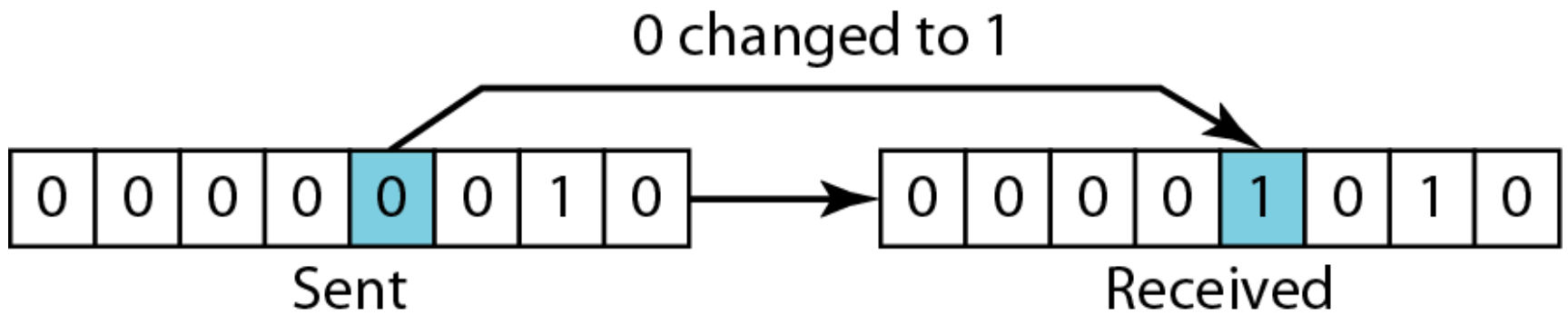
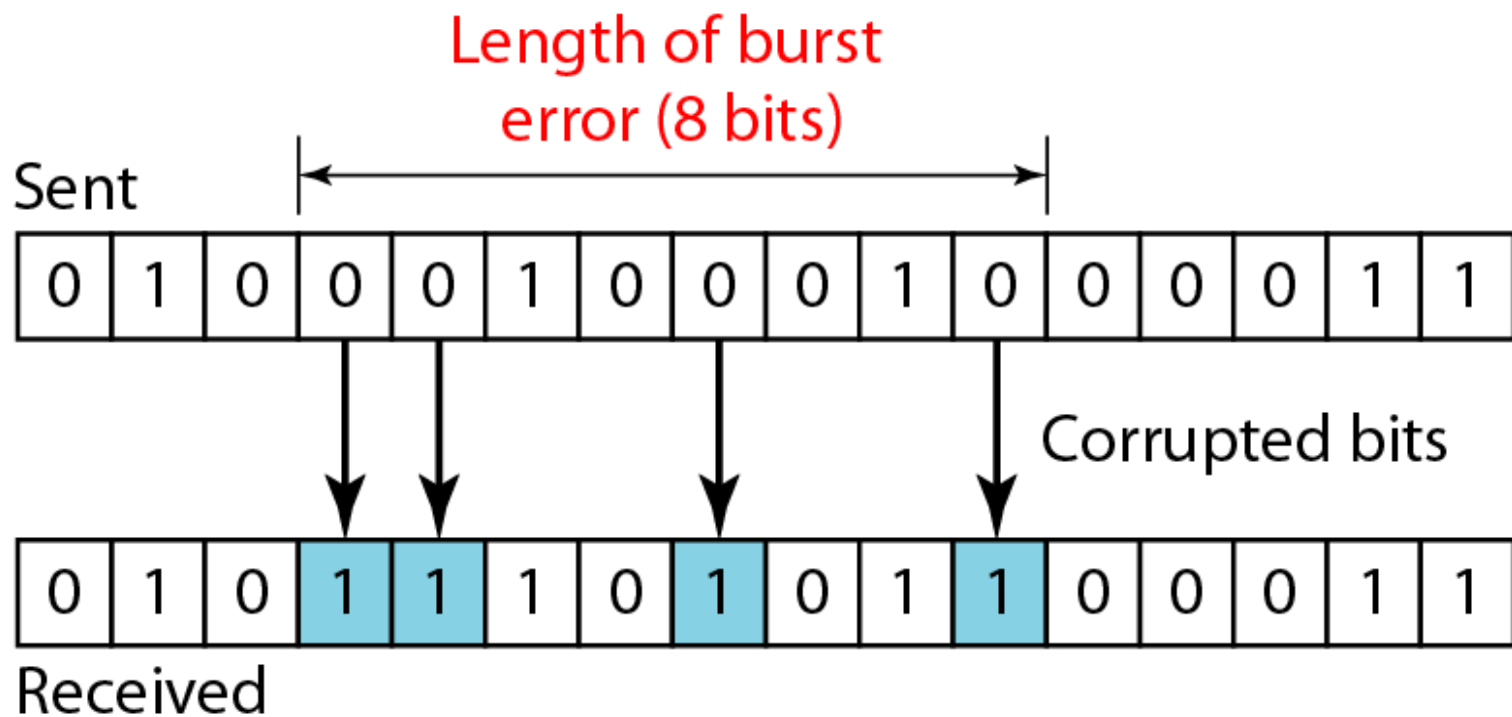


Single-bit error



Burst error of length 8



The most popular Error Detecting Techniques are:

- Single parity check
- Two-dimensional parity check
- Checksum
- Cyclic redundancy check

Detection -example

At the sender side

Suppose the sender wants to send the word *world*. In ASCII the five characters are coded as

1110111 1101111 1110010 1101100 1100100

The following shows the actual bits sent

11101110 11011110 11100100 11011000 11001001

At the receiver side (Possibility-1)

Now suppose the word world in the Example is received by the receiver without being corrupted in transmission.

11101110 11011110 11100100 11011000
11001001

The receiver counts the 1s in each character and comes up with even numbers (6, 6, 4, 4, 4). The data is accepted.

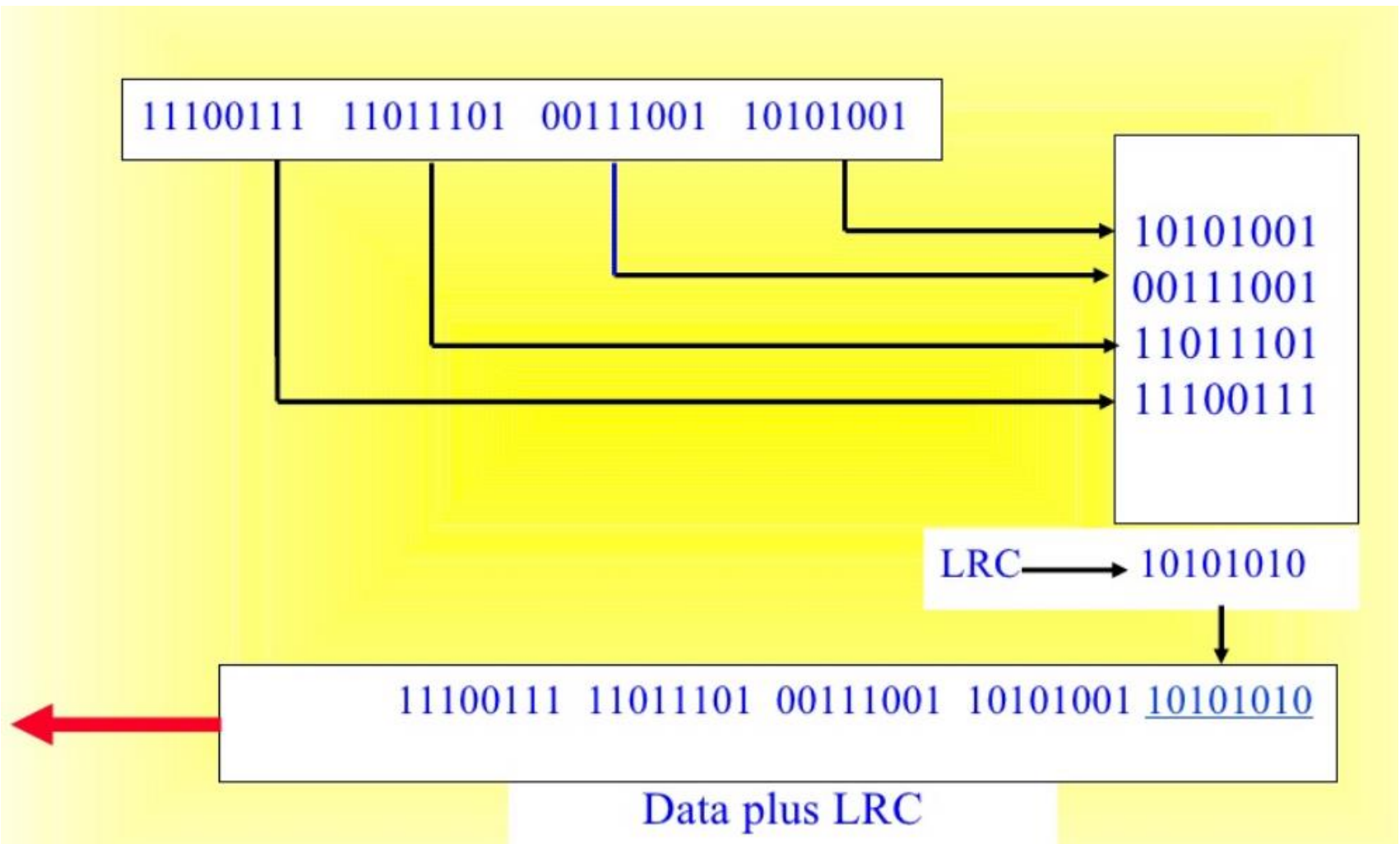
At the receiver side (Possibility-2)

Now suppose the word world in the Example is corrupted during transmission.

11111110 11011110 11101100 11011000
11001001

The receiver counts the 1s in each character and comes up with even and odd numbers (7, 6, 5, 4, 4). The receiver knows that the data are corrupted, discards them, and asks for retransmission.

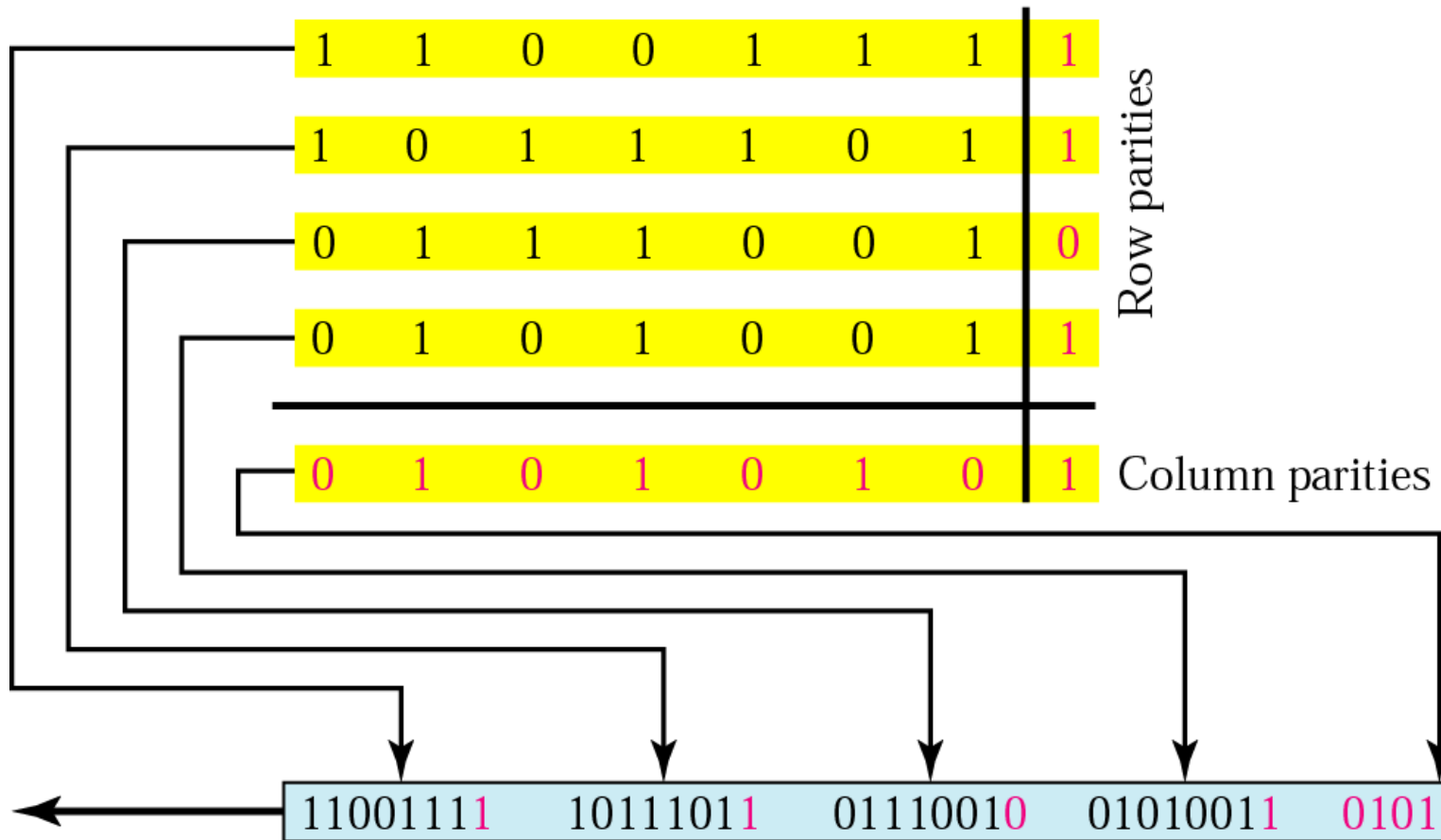
Longitudinal Redundancy Check LRC



Two Dimensional Parity

Original data

1100111	1011101	0111001	0101001
---------	---------	---------	---------



1100111	1	1011101	1	0111001	0	0101001	1	0101010	1
---------	---	---------	---	---------	---	---------	---	---------	---

Data and parity bits

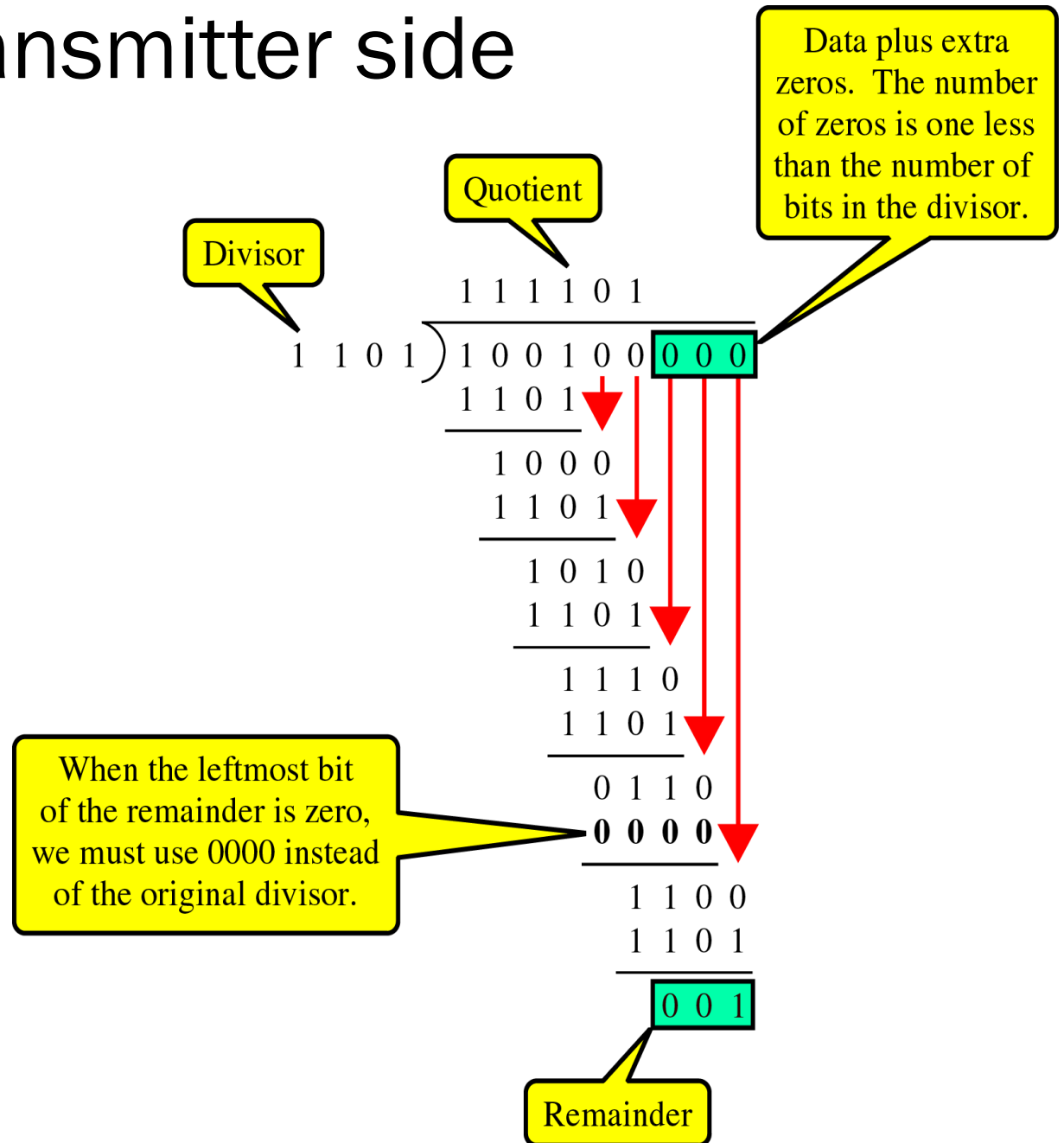
- Find the 2D parity for the data blocks

11100111 11011101 00111001 10101001

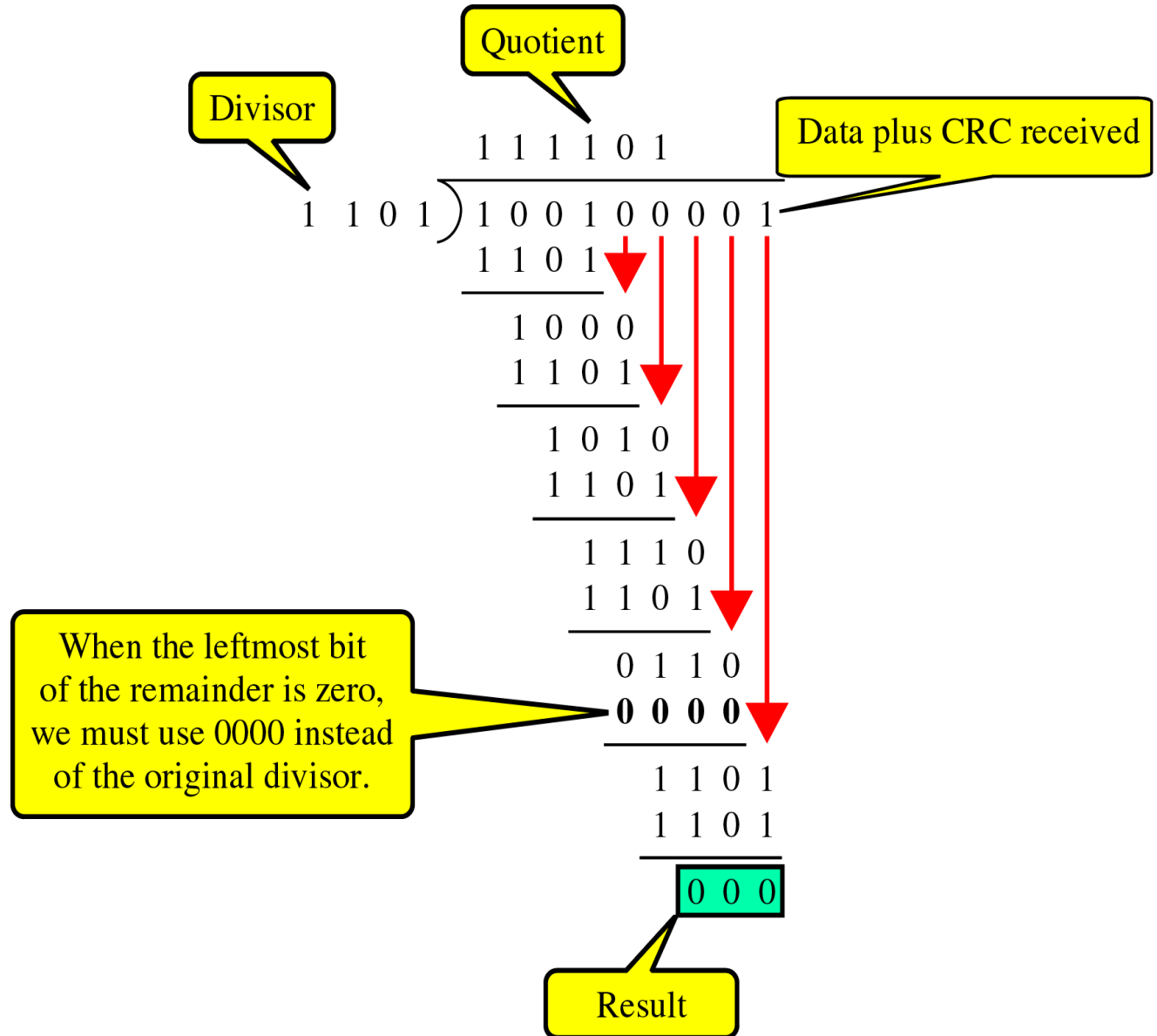
At the transmitter side

- CRC generator

~ uses modular-2 division.



At the receiver side



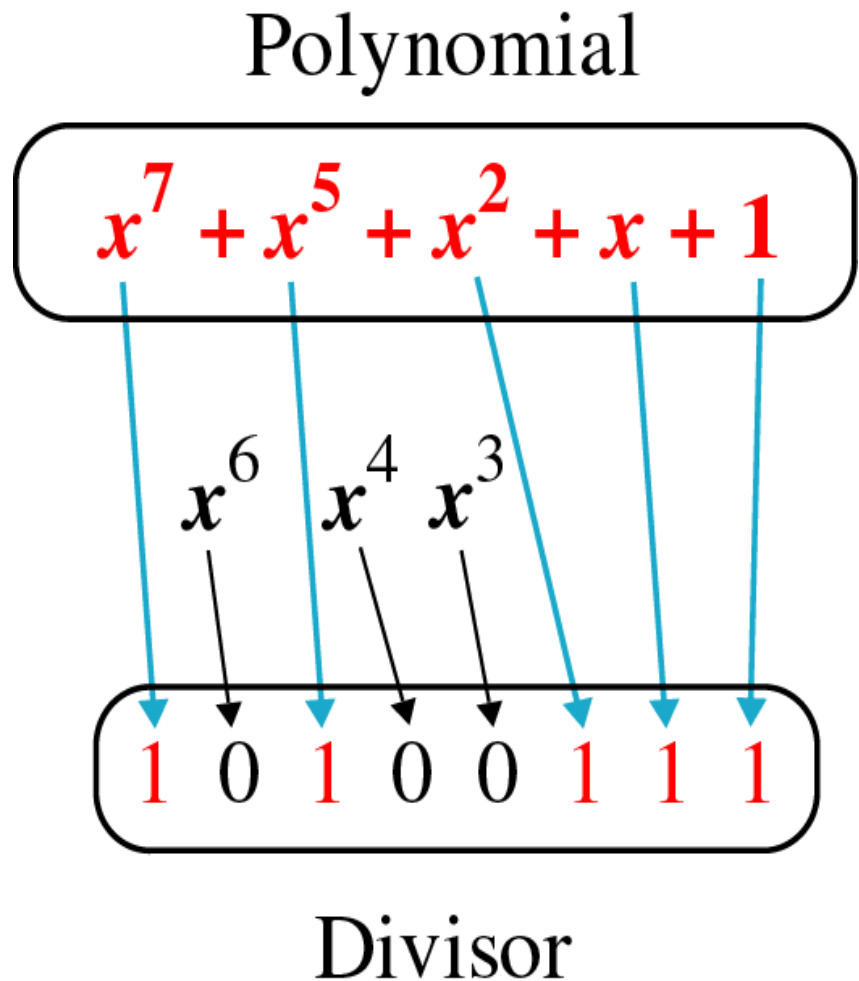
CRC

- Polynomials
 - CRC generator(divisor) is most often represented not as a string of 1s and 0s, but as an algebraic polynomial.

$$x^7 + x^5 + x^2 + x + 1$$

CRC (cont'd)

- A polynomial representing a divisor



CRC (cont'd)

- Standard polynomials

CRC-12

$$x^{12} + x^{11} + x^3 + x + 1$$

CRC-16

$$x^{16} + x^{15} + x^2 + 1$$

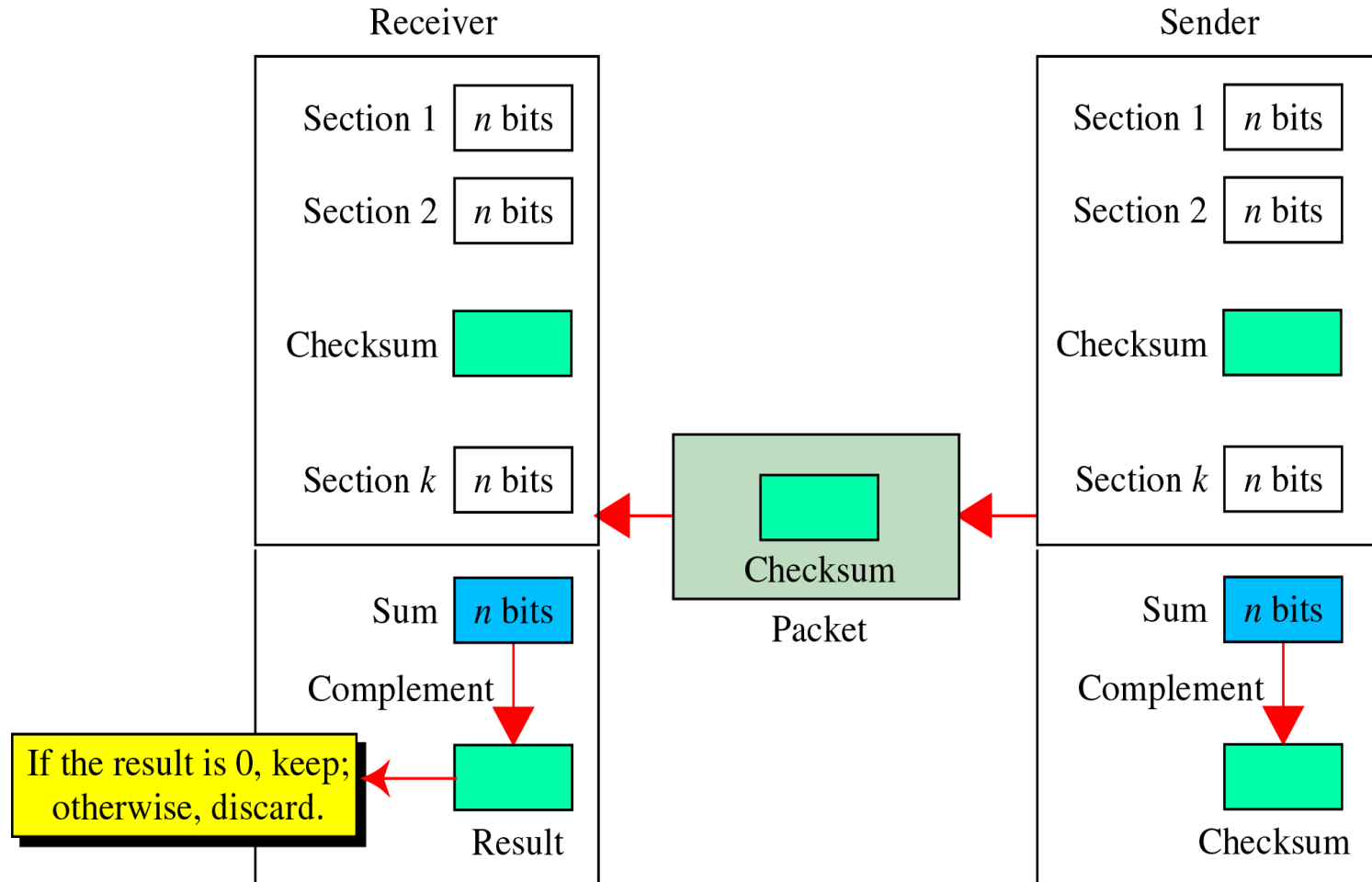
CRC-ITU-T

$$x^{16} + x^{12} + x^5 + 1$$

CRC-32

$$x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$$

Checksum



Checksum

- To create the checksum the sender does the following:
 - The unit is divided into K sections, each of n bits.
 - Section 1 and 2 are added together using one's complement.
 - Section 3 is added to the result of the previous step.
 - Section 4 is added to the result of the previous step.
 - The process repeats until section k is added to the result of the previous step.
 - The final result is complemented to make the checksum.

Checksum (at the sender side)

Original data : 10101001 00111001

10101001

00111001

11100010 Sum

00011101 Checksum

10101001 00111001 00011101

Checksum (at the receiver side)

Received data : 10101001 00111001 00011101

10101001

00111001

00011101

11111111 ← Sum

00000000 ← Complement